

Heuristic Drone Routing for Pheromone Delivery

3 November 2025

Team Collective Solutions

Yana Padte <ypadt001@ucr.edu>

Saumya Chourasia <schou065@ucr.edu>

Anokhee Shah <ashah174@ucr.edu>

Allen Biju <abiju009@ucr.edu>

Parth Awade <pawad006@ucr.edu>

Abstract

Our client, Mr. Keogh, requested a system to improve the efficiency of a pheromone-delivery drone used to control the navel orangeworm infestation on his almond farms. The drone disperses pheromone paste to disrupt mating behavior and reduce crop damage sustainably, but its limited flight time makes route optimization essential. To solve this, we implemented three algorithms: Random Search, Nearest Neighbor, and Augmented Nearest Neighbor. The system reads coordinate files for target locations, computes and displays the optimal path, total distance, and runtime, and supports user interruption at any point. The program also creates a visualization for the path generated. Testing shows that the anytime algorithms progressively deliver shorter routes and faster results, allowing the drone to accomplish its mission more quickly while using less battery power.

Elicitation

Initial Group Meeting: October 2, 2025

Submission: October 3, 2025 at 4:51 pm

Key Questions:

1. Do different locations require different amounts of pheromones, or is each visit identical in terms of pheromone quantity and cost?
2. Does the drone have enough pheromone capacity to hit/cover all stations in one journey?
3. Is minimizing the drone's travel distance the ONLY goal?
4. Is there anything else we forgot to mention/ask that is relevant?

Clarifications Received: Each location is identical; a single journey is assumed feasible; minimizing distance is the primary goal; and updates are desired at any time.

Final Understanding: The system should compute optimal routes for up to 256 drone locations using an anytime approach, output the best route and distance, and handle user interruptions gracefully.

Requirement Specification

We begin with Table 1, which lists all acronyms used.

CLI	Command Line Interface
BSF	Best-So-Far
NN	Nearest Neighbor
Aug-NN	Augmented Nearest Neighbor
I/O	Input/Output
CPU	Central Processing Unit
TXT	Text File
JPEG	Image File

Table 1: Acronym List

3.1 System Context Diagram

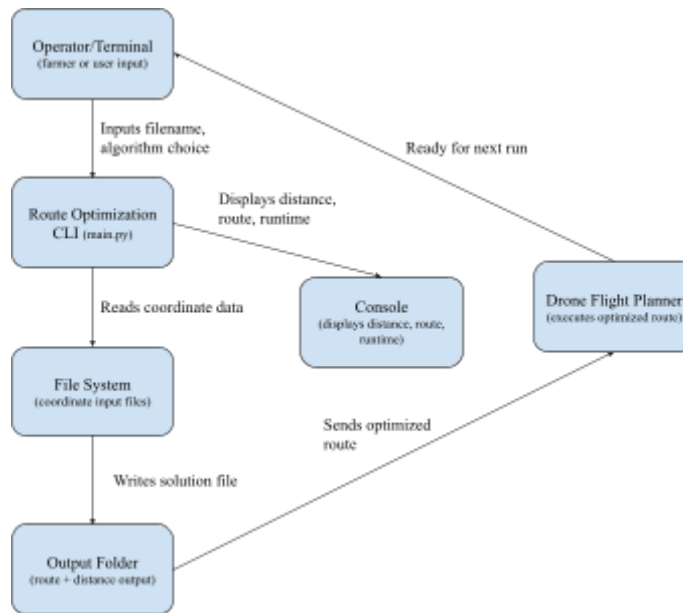


Figure 1: Running on the terminal, the Route Optimization CLI reads from farm coordinate files, then lets the operator choose an algorithm. It then outputs the best route, distance, and runtime. It interacts with the file system for inputs, the console for results, and saves a solution route file.

3.2 System Functions

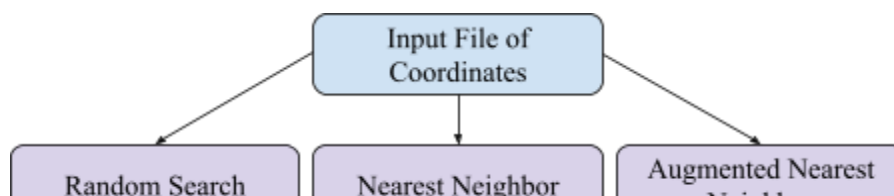


Figure 2: Illustrated above is the workflow of the route optimization system. The input file with the different coordinates is processed using one of the three algorithms: Random Search, Nearest Neighbor, and Augmented Nearest Neighbor. Each algorithm then outputs the best route that it can find as well as the total distance, and runtime for performance comparison.

The drone supports the following primary system functions:

1. Verify the format of the input file with the complete list of drone locations.
2. Parse the file to populate an array of drone location coordinates.
3. Ensure the number of nodes does not exceed 256.
4. Execute one of three algorithms to compute drone routes (Random Search, Nearest Neighbor, Augmented Nearest Neighbor).
5. Provide anytime updates on the shortest/best distance so far, with user interruption via the ENTER key.
6. Compute route distances using the Euclidean Distance formula.
7. Output the best route, total distance, and runtime to the console and to a separate solution text file, and warn the user if the computed route exceeds 6000 meters.

3.3 Operating Environment

- Standard laptop using Python
- Local file access only
- Dependencies limited to Python standard libraries, with numpy support for visualization

3.4 Design & Implementation Constraints

- Maximum of 256 locations/nodes per input file.
- The algorithm must terminate gracefully when the user interrupts it with the ENTER key.
- Input and output files use fixed numeric formats; the input file contains (x, y) coordinates, and the output file contains the best route.
- Euclidean distance is used to compute travel distances.
- Max Distance is 6000 meters.

3.5 Assumptions and Dependencies

- Input always valid numeric data
- Interrupt triggered by ENTER.
- Distance metric = Euclidean.
- File I/O is local.

Design

4.1 Workflow Summary

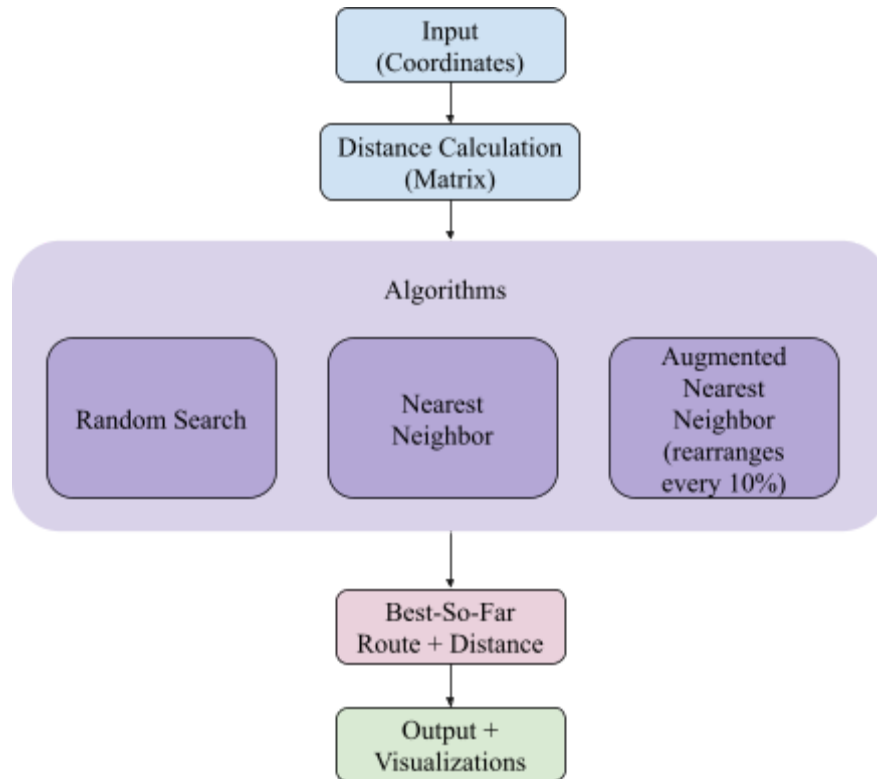


Figure 3: This diagram shows the overall process of the Pheromone Delivery System. The program reads an input file containing coordinates of the stations, and performs a distance calculation with one of the three chosen algorithms: Random Search, Nearest Neighbor, or Augmented Nearest Neighbor. Afterwards, it outputs the Best-So-Far route, distance, runtime, and a visualization of the route.

The anytime loop we implemented allows our algorithm to search for better routes while continuously providing intermediate results. The system monitors user input through a separate thread that waits for the ENTER key to be pressed, using a flag implemented in each function. This design ensures that the program can terminate at any moment, returning the best route found so far without losing progress or overloading the CPU. Random Search samples routes randomly to provide a baseline distance, Nearest Neighbor constructs a route sequentially visiting the nearest unvisited node, and Augmented Nearest Neighbor improves the Nearest Neighbor route by performing random swaps and shuffling segments of the route while outputting the intermediate best distance while it is running.

Each algorithm operates independently on the same input data files but uses different strategies to generate solutions. Random Search explores the solution space broadly, giving a low-effort estimate using a random algorithm. Nearest Neighbor quickly provides a deterministic route that is often better than random sampling, but there is only one consistent output. By utilizing local optimization and random shuffling with a 10% chance, Augmented Nearest Neighbor improves our Nearest Neighbor function's

result by calling it within its function. The system is guaranteed to have a strong initial solution and a practical algorithm for ongoing improvement, thanks to this combination of nearest-neighbor algorithms.

The distance calculation component reads data from the input coordinate file and computes the Euclidean distance between nodes. The chosen nearest-neighbor algorithms use these distances as input, iteratively improving the output route and updating the best-so-far solution. After that, the console is updated with the route, total distance, and runtime. Additionally, a different solution file made in our program contains the route. We also implemented visualization to provide the user with an intuitive view of the drone's path for all three algorithms.

Insight

We began with the Random Search algorithm because its simple functionality provided a clear performance baseline. It explores routes entirely at random and records the best result found so far. Although inefficient, this approach helped ensure that the algorithms we develop are practical, since they should be able to outperform Random Search if implemented correctly.

We then implemented the Nearest Neighbor algorithm as a greedy heuristic that always selects the closest unvisited node. This drastically improves runtime and distance compared to the Random Search algorithm, though it can get trapped in locally optimal routes. To address that barrier, we implemented the Augmented Nearest Neighbor, which builds on the Nearest Neighbor algorithm by introducing swaps and route shuffling, combining a deterministic and a random approach. Finally, the early-abandon feature allows any algorithm to be interrupted by the user at any time, returning the best route found so far. Together, these elements form an anytime algorithm that delivers increasingly better routes the longer it runs.

Implementation

Our project was implemented in Python and divided into four files: `main.py`, `randomSearch.py` (for the Random Search algorithm), `nearestNeighbor.py` (for the Nearest Neighbor algorithm), and `augmented.py` (for the Augmented Nearest Neighbor algorithm). The `main.py` file is the starting point for our program, loading the coordinate files and asking the user to select which search algorithm they would like to use. The three algorithms are split into different files for clarity.

The ENTER interruption is handled through a background thread that monitors user input, allowing the algorithm to stop and return the best route found so far cleanly. Each algorithm outputs its runtime and best route to the terminal and writes the final route to an output file named using a specified format.

Visualization of the computed routes is implemented using Matplotlib. After computing the best route and distance, the program converts the ordered coordinates into x-y pairs, and plots them as a connected path. The starting/ending location is marked in red. The resulting image is saved automatically as a JPEG file named according to the dataset and solution distance.

Github Link - <https://github.com/saumyaxc/Pheromone-Delivery-Drone/tree/main>

6.1 Naming Convention of Solution Files

→ **Best Route Solution:** <filename>_SOLUTION_<ceiling value of best distance>.txt

→ **Visualization of Best Route:** <filename>_SOLUTION_<ceiling value of best distance>.jpeg

6.2 Algorithm Traces

This section shows the outputs and route visualizations for all three implemented algorithms using the same dataset (32Almonds.txt). Each case demonstrates the algorithm's runtime, route file output, and resulting path visualization.

Case 1 – Random Search

Inputs: 32Almonds.txt, 1, *ENTER*

```
Enter the name of the file: 32Almonds.txt
```

```
There are 32 nodes.
```

```
Choose an algorithm to solve the Pheromone Delivery Drone problem.
```

1. Random Search
2. Nearest Neighbor
3. Augmented Nearest Neighbor

```
Enter choice: 1
```

```
Running Random Search Algorithm...
```

```
Press ENTER if you want to stop the search!
```

```
Shortest Route Discovered So Far:
```

```
1628.9  
1551.8  
1372.0  
1347.6  
1299.0  
1270.6  
1255.1  
1240.0  
1216.7  
1169.9  
1126.0  
1113.0  
1107.0
```

```
1099.1
```

```
1066.0
```

```
Best Distance: 1066 meters
```

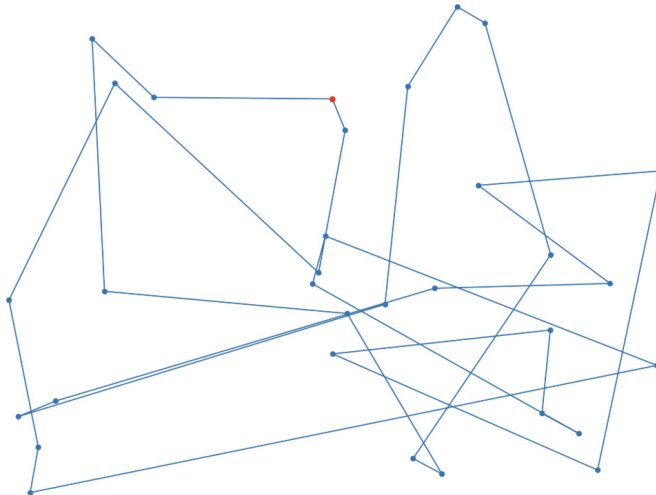
```
Best Route: [1, 10, 28, 14, 23, 29, 5, 25, 27, 20, 4, 8, 21, 24, 32, 3, 16,  
15, 1, 17, 26, 7, 6, 18, 13, 31, 19, 22, 12, 9, 2, 30, 1]
```

```
Runtime: 162.7502 seconds
```

```
==> Route written to disk as 32Almonds_SOLUTION_1066.txt
```

```
==> Visualization saved as 32Almonds_SOLUTION_1066.jpeg
```

Visualization of Best Route:



Case 2 – Nearest Neighbor

Inputs: 32Almonds.txt, 2, *ENTER*

```
Enter the name of the file: 32Almonds.txt
```

```
There are 32 nodes.
```

```
Choose an algorithm to solve the Pheromone Delivery Drone problem.
```

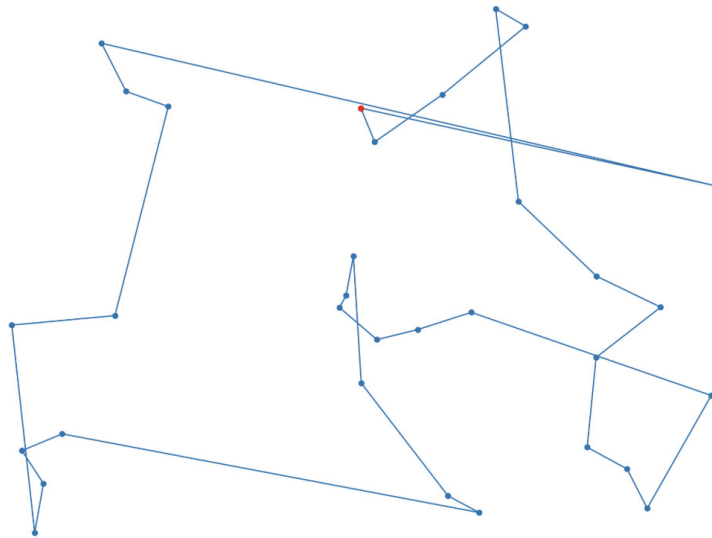
1. Random Search
2. Nearest Neighbor
3. Augmented Nearest Neighbor

```
Enter choice: 2
Running Nearest Neighbor Algorithm...

Best Distance: 616 meters
Best Route: [1, 30, 4, 27, 20, 16, 25, 3, 26, 7, 6, 11, 31, 32, 8, 23, 18,
2, 13,
17, 5, 29, 24, 21, 22, 19, 12, 14, 10, 9, 28, 15, 1]
Runtime: 0.0013 seconds

==> Route written to disk as 32Almonds_SOLUTION_616.txt
==> Visualization saved as 32Almonds_SOLUTION_616.jpeg
```

Visualization of Best Route:



Case 3 – Augmented Nearest Neighbor

Inputs: 32Almonds.txt, 3, *ENTER*

```
Enter the name of the file: 32Almonds.txt

There are 32 nodes.

Choose an algorithm to solve the Pheromone Delivery Drone problem.
1. Random Search
2. Nearest Neighbor
```


3. Augmented Nearest Neighbor

Enter choice: 3

Running Augmented Nearest Neighbor Algorithm...

Press ENTER if you want to stop the search!

Running augmented nearest neighbor (press ENTER to stop)...

Shortest Distance So Far:

615.55

614.36

591.87

565.72

562.63

555.6

550.8

549.91

546.6

537.81

534.12

Best Distance: 535 meters

Best Route: [1, 30, 20, 27, 16, 15, 25, 3, 31, 11, 6, 7, 26, 32, 8, 13, 2, 18, 23,

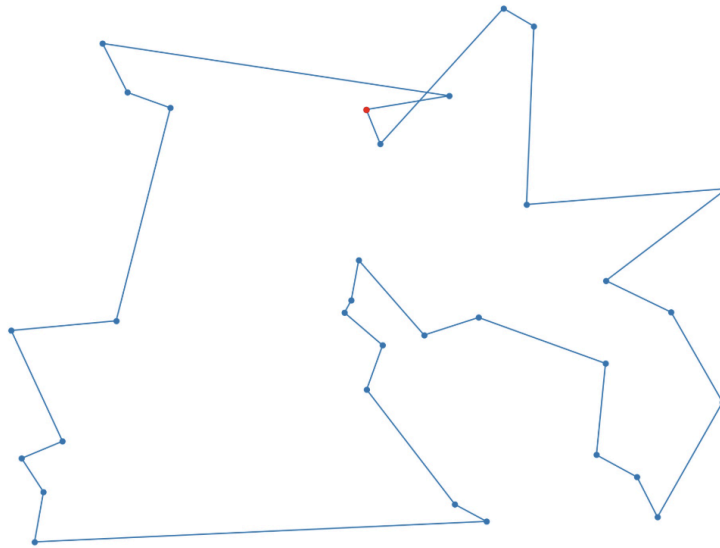
17, 5, 29, 19, 22, 21, 24, 12, 14, 10, 9, 28, 4, 1]

Runtime: 450.8248 seconds

=> Route written to disk as 32Almonds_SOLUTION_535.txt

=> Visualization saved as 32Almonds_SOLUTION_535.jpeg

Visualization of Best Route:



Testing

7.1 Testing for Correct Program Behavior

To handle incorrect input and user errors, we added specific functionality to ensure our program could handle any issues and work as expected.

Case 1 – Missing file

→ **Condition:** Input file contains unreadable or non-numeric values

→ **Expected Behavior:** The program catches the error and prints a warning message

```
except FileNotFoundError:
    print(f"Error: The file '{filename}' does not exist. Please try
again.\n")
```

Case 2 – Incorrect Formatting (not in x,y format)

→ **Condition:** Input file contains lines that do not have exactly two numeric values

→ **Expected Behavior:** The program raises an error and stops execution

```
for line in file:
    if len(line.split()) != 2:
        raise ValueError("Error in file formatting! Every line needs two
```

```
numbers")
    x, y = map(float, line.split())
    coordinates.append((x, y))
```

Case 3 – Too Many Coordinates (N > 256)

→ **Condition:** Input file contains lines that do not have exactly two numeric values

→ **Expected Behavior:** The program raises an error and stops execution

```
if len(coordinates) > 256:
    print("The number of nodes exceeds 256! Try again with fewer nodes.")
    sys.exit()
```

Case 4 – Empty File

→ **Condition:** Input file is empty or contains no valid coordinates

→ **Expected Behavior:** The program exits with an appropriate message

```
if len(coordinates) == 0:
    print("No coordinates found in file")
    sys.exit()
```

Case 5 – Corrupted or Invalid Values

→ **Condition:** Input file contains unreadable or non-numeric values

→ **Expected Behavior:** The program catches the error and prints a warning

```
except ValueError as e:
    print(f"Value is not recognized: {e}. Please try again.\n")
```

Case 6 – Negative Coordinates

→ **Condition:** Input file contains coordinates with negative x or y values

→ **Expected Behavior:** The program raises an error and stops, since values must be positive

```
if x < 0 or y < 0:
    raise ValueError("Coordinate values cannot be negative.")
```

Case 7 – Duplicate Coordinates

→ **Condition:** Input file contains repeated (x, y) coordinate pairs

→ **Expected Behavior:** The program identifies the duplicate, prints a warning, and skips over it

```
if (x, y) in coordinates:
    raise ValueError("Duplicate coordinate found. Skipping over")
    continue
```

7.2 Quality of Solution

We tested our Random Search, Nearest Neighbor, and Augmented Nearest Neighbor algorithms on 5 different datasets. We used the Random Search as a baseline method for this project, which visited all stations at random and gave us the longest route. Nearest Neighbor essentially had a significantly reduced total distance by greedily visiting the closest unvisited node, while Augmented Nearest Neighbor would further refine the routes through the recurring arrangements. Adding on the Augmented Nearest Neighbor rearranges segments every 10% of runtime to reduce total distance. These improvements are more noticeable on the larger datasets such as 128Hazelnut and 256Cashew where the Augmented Nearest Neighbor was able to achieve up to 500 - 600 meters shorter routes than Random Search, which displays the scalability and robustness.

	Random Search	Nearest Neighbor	Augmented Nearest Neighbor
32Almonds	1026.83 m	616 m	539 m
64Walnuts	2325.67 m	816 m	751.25 m
128Circle201	4117 m	201 m	201 m
128Hazelnut	5267.4 m	1085 m	1044.75 m
256Cashew	11456.6 m	1542 m	1490.2 m

Table 2: Average routes lengths achieved by each algorithm across the five data sets

Additionally, in order to test the optimality of our augmented algorithm, we ran a test that shows how the solution quality improves over time. The total distance values are taken during execution and monitored as new, more efficient distances were calculated. As the routes get better over time, the convergence graphs show their performance.

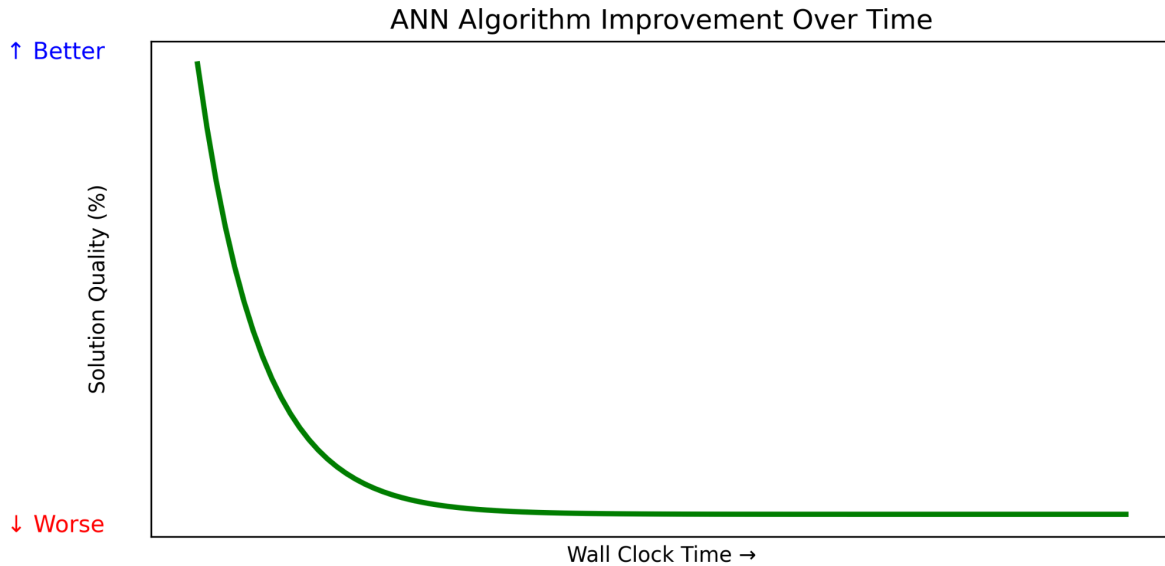


Figure 4: Augmented Nearest Neighbor over a 30 minute period

7.3 Algorithm Performance

From the results, it is evident that the Random Search algorithm performed poorly overall. The average number of tour distances increased very quickly as the number of nodes increased. It rises from 1000 meters at 32 nodes to over 11,000 meters at 256 nodes. This shows that there is a lack of heuristic approach which leads the algorithm to explore paths that are inefficient as the problem of the size grows. In contrast to this, the two algorithms that use an heuristic approach: Nearest Neighbor and Augmented Nearest Neighbor had shorter and more stable routes across the different data sizes. The tour distance remains below 1000 meters, which shows a linear growth when the number of nodes increases.

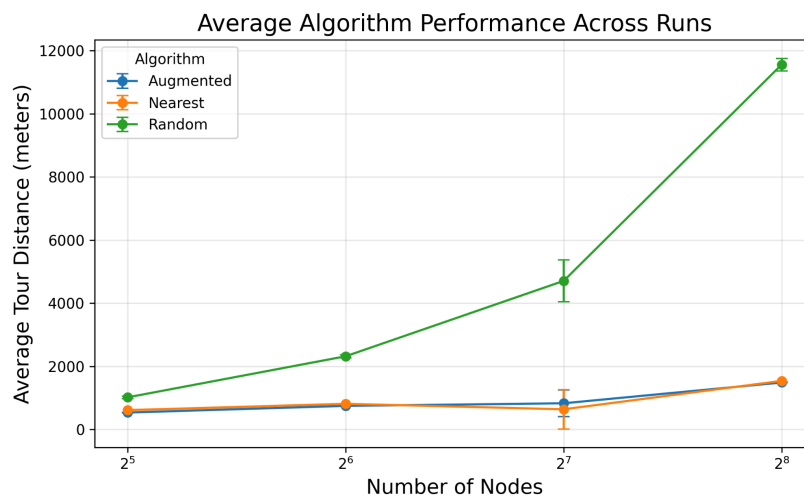


Figure 5: This graph shows the average algorithm performance for the three algorithms: Random Search, Nearest Neighbor, and Augmented Nearest Neighbor

Final Test

To validate the algorithm performance under more realistic conditions, additional tests were run on the Augmented Nearest Neighbor algorithm on the 5 datasets: 32Almonds.txt, 64Walnut.txt, 128Circle201.txt, 128Hazelnut.txt, 256Cashew.txt. The algorithms were interrupted at 1, 10, and 100 seconds in order to evaluate how route quality evolves with longer computational times.

	Interrupted at 1 Sec	Interrupted at 10 Sec	Interrupted at 100 Sec
32Almonds.txt	538.17 m	535.53 m	538.4 m
64Walnut.txt	764.67 m	749.63 m	746.3 m
128Circle201.txt	201 m	201 m	201 m
128Hazelnut.txt	1082.23 m	1065.97 m	1038.13 m
256Cashew.txt	1547.71 m	1534.98 m	1499.03 m

Table 3: Results of the Augmented Nearest Neighbor algorithm on five datasets interrupted at 1, 10, and 100 seconds

After doing multiple runs the averages were computed to obtain the final distances for each dataset at the interrupted time. From this table the observations that were made is that the smaller dataset such 32Almonds and 64Walnut the route length essentially stabilized more quickly and had very minimal improvement between 1second and 100 seconds which indicates a rapid convergence. In contrast to this the larger datasets such as 128Hazelnut and 256Cashew show a continuous improvement as the runtime increased. Specifically, the 256Cashew dataset improved from 1547.71 meters at 1 second to 1499.03 meters at 100 seconds. This indicates that additional runtime allowed the algorithm to reduce any redundant traversal and explore better routes.

Conclusion

In this project we have implemented an anytime algorithm that was designed to improve solution quality as it runs for a longer time and provides reliable performance under different time constraints. The two advanced algorithms (Nearest Neighbor and Augment Nearest Neighbor) consistently outperform the baseline algorithm (Random Search), confirming that heuristic decision-making improves route search efficiency. The complete implementation meets all functional constraints, including input validation and execution time limits, and maintains stability across different input sizes.

References

- Keogh, P1 Detailed Breifing.pptx
- Keogh, Sample_of_P1.pdf
- <https://docs.python.org/3/library/threading.html>
- <https://machinelearningmastery.com/tutorial-to-implement-k-nearest-neighbors-in-python-from-scratch/>
- <https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/>
- <https://machinelearningmastery.com/hyperparameter-optimization-with-random-search-and-grid-search/>
- https://www.w3schools.com/python/python_file_open.asp
- <https://www.geeksforgeeks.org/artificial-intelligence/heuristic-search-techniques-in-ai/>
- <https://www.geeksforgeeks.org/dsa/greedy-best-first-search-algorithm/>
- <https://www.geeksforgeeks.org/machine-learning/approximate-nearest-neighbor-ann-search/>
- https://www.w3schools.com/python/matplotlib_plotting.asp